COL1000: Lab 11

Semester 2, 2025–26

Instructions Upload your code to Gradescope as: `<entryNumber>_q<quesNumber>.py` unless stated otherwise. After uploading, run the autograder to check that all tests pass. Make sure not to include any custom prompt text in `input()` calls, and any extra text in `print()` statements. You must not use the `math, ast, intertools, sympy` libraries.

Question 1: Spellathon Spellathon is a word-puzzle game where players must form as many valid words as possible, each at least four letters long, using a specific set of seven provided letters. The game tracks the player's score based on unique correct guesses and identifies any missed words from the valid set once the attempts are exhausted.

Implement a `Spellathon` class in a file `spellathon_q1.py` that can be used as a module for the following code. The class constructor should initialize from a list `data` where `data[0]` contains the 7 game letters and the remainder forms a set of valid words. The class must include a `play(word)` method that increments an internal attempts counter and returns '`Correct guess!`' if the word is at least 4 letters long, present in the valid word list, and not previously found; otherwise, it must return '`Incorrect guess!`'. You must ensure that score is an integer counting the number of correct guesses, and `remaining_words` is a sorted list of all missed valid words.

```
from spellathon_q1 import Spellathon

data = ['codlouw', 'clod', 'cloud', 'cold', 'cool', 'could', 'cowl', 'loud', '
    ↪ wood', 'wool', 'would']

game = Spellathon(data)
print('Use the letters:', data[0].upper(), 'to make words of minimum 4 letters
    ↪ . You have 10 attempts.')

for i in range(10):
    word = input('Enter the word (in lower case): ')
    message = game.play(word)
    print(message) # 'Incorrect guess!' or 'Correct guess!'

print('Score:', game.score)
print('Missed words in the sorted order:', game.remaining_words)
```

Question 2: University Registration and Transcripts Complete the starter code below to implement a university course management system. The main program reads a sequence of commands from standard input (`sys.stdin`). It will manage registrations, read marks from external files, and generate individual transcript files.

The system must follow these rules:

- **Registration:** A student can register for a course if it has not reached its maximum

capacity. If the course has reached its capacity, print: `Course full. <name> cannot enroll in <coursecode>.` to standard output.

- **Dropping:** If an enrolled student drops a course, they are removed from the course's registry, freeing up a spot for capacity checks.
- **Marks Loading:** The `MARKS <coursecode>` command must open a file named `<coursecode>_marks.txt` (e.g., `COL1000_marks.txt`). This file contains space-separated lines of `entrynum` and `marks`. It should record these marks for enrolled students. You may assume that marks will be in the range $[0, 100]$.
- **Transcripts:** The `TRANSCRIPT <entrynum>` command must generate a text file named `<name>_<entrynum>_transcript.txt` (e.g., `Bob_2025TT13301_transcript.txt`). The file should list all courses the student, sorted alphabetically by `coursecode`, formatted as `<coursecode> <marks>` on each line. If marks are missing, print `None`. The last line must have credit-weighted CGPA (format: `<CGPA> <int(cgpa_val)>`) computed using this 10-point grade system: $>= 90 : A$, $>= 80 : A-$, $>= 70 : B$, $>= 60 : B-$, $>= 50 : C$, $>= 40 : C-$, $>= 30 : D$, $< 30 : 0$. Note that courses with `None` marks are excluded from the CGPA calculation.

```python
import sys

class Student:
    def __init__(self, entrynum, name):
        self.entrynum = entrynum
        self.name = name
        self.current_courses = [] # List of Course objects

    def generate_transcript(self):
        # TODO: Create a file named <name>_<entrynum>_transcript.txt
        pass

class Course:
    def __init__(self, coursecode, credits, capacity):
        self.coursecode = coursecode
        self.credits = credits
        self.capacity = capacity
        self.registered_students = {} # entrynum -> marks

    def add_student(self, student_obj):
        # TODO: Handle capacity check and enrollment
        # Print error message to stdout if course is full
        pass

    def remove_student(self, student_obj):
        # TODO: Remove student from the course if they are enrolled.
        pass

    def load_marks(self):
```

```
        # TODO: Open <coursecode>_marks.txt and read marks
        # Read data in format: entrynum marks
        pass

class University:
    def __init__(self):
        self.students = {} # entrynum -> Student
        self.courses = {} # coursecode -> Course

    def process_commands(self):
        for line in sys.stdin:
            parts = line.strip().split()
            if not parts: continue

            cmd = parts[0]
            if cmd == 'COURSE':
                code, creds, cap = parts[1], int(parts[2]), int(parts[3])
                self.courses[code] = Course(code, creds, cap)
            elif cmd == 'ENROLL':
                entrynum, name = parts[1], parts[2]
                self.students[entrynum] = Student(entrynum, name)
            elif cmd == 'REGISTER':
                entrynum, code = parts[1], parts[2]
                self.courses[code].add_student(self.students[entrynum])
            elif cmd == 'DROP':
                entrynum, code = parts[1], parts[2]
                self.courses[code].remove_student(self.students[entrynum])
            elif cmd == 'MARKS':
                code = parts[1]
                self.courses[code].load_marks()
            elif cmd == 'TRANSCRIPT':
                entrynum = parts[1]
                self.students[entrynum].generate_transcript()

uni = University()
uni.process_commands()
```

Example Standard Input:

```
COURSE COL1000 4 2
COURSE MTL2000 4 3
ENROLL 2025CS52201 Alice
ENROLL 2025TT13301 Bob
ENROLL 2025AM14405 Charlie
REGISTER 2025CS52201 COL1000
REGISTER 2025TT13301 COL1000
REGISTER 2025AM14405 COL1000
DROP 2025CS52201 COL1000
```

```
REGISTER 2025AM14405 COL1000
REGISTER 2025TT13301 MTL2000
MARKS COL1000
TRANSCRIPT 2025TT13301
```

Example Content of `COL1000_marks.txt`:

```
2025TT13301 70
2025AM14405 72
```

Expected Standard Output:

```
Course full. Charlie cannot enroll in COL1000.
```

Expected Created File (`Bob_2025TT13301_transcript.txt`):

```
COL1000 70
MTL2000 None
CGPA 8
```

Question 3: Sliding Tile Game 128 is a popular single-player sliding-tile game played on a 4×4 grid. The player uses arrow keys (or U/D/L/R inputs) to slide all tiles in a chosen direction, combining matching numbers. When two tiles of the same value collide, they merge into one tile with double the value. After each valid move, a new tile with value 2 appears in a random empty cell. The game rules are:

- **Movement:** All tiles slide as far as possible in the chosen direction until they hit an edge or another tile.
- **Merging:** When two tiles with equal values collide, they merge into a single tile with their sum. **Note:** Each tile position can merge only once per move (e.g., `[2, 2, 2, 2]` in moving left becomes `[4, 4, 0, 0]`).
- **New Tiles:** After each move, the value of a random empty cell (storing 0) is converted to 2.
- **Game States:** A win is triggered by a tile reaching 128. The game is lost when the grid is full *and* no adjacent tiles have equal values (i.e. no moves are possible).

You can read about it here and play it here.

Implement a `Game` class in a file `game128_q3.py` that can be used as a module to manage the core logic of a 128 game. The class constructor should initialize a 4×4 grid provided as a list of lists. The class must include the following methods:

- `add_randomness(r, c, val)`: Updates the grid at row `r` and column `c` with the given `val`.
- `move_left()`, `move_right()`, `move_up()`, and `move_down()`: Each method must slide all non-zero tiles fully in the chosen direction and merge adjacent tiles of equal value. Each method must update the grid in place.
- `check()`: Returns `1` if any tile in the grid has a value of 128 (indicating a win); else returns `-1` if no move is possible (indicating game is over without a win), else `0`.

The main script below demonstrates how your module will be used. It handles user input, random tile generation, and displaying the grid.

```
import random
from game128_q3 import Game

def spawn_tile(game):
    empty_cells = []
    for r in range(4):
        for c in range(4):
            if game.grid[r][c] == 0: empty_cells.append((r, c))
    if empty_cells:
        r, c = random.choice(empty_cells)
        game.add_randomness(r, c, 2)

def print_grid(grid):
    for row in grid:
        for val in row:
            print(val, end='\t')
        print()
    print()

# Your module will be tested against arbitrary starting configurations as well
initial_grid = [[0]*4, [0]*4, [0]*4, [0]*4]
game = Game(initial_grid)
print('Welcome to 128! Reach the 128 tile to win.')

while True:
    spawn_tile(game)
    print_grid(game.grid)

    if game.check() == -1:
        print('Game over.')
        break

    move = input('Enter a valid move (U/D/L/R): ').upper()

    if move == 'U': game.move_up()
    elif move == 'D': game.move_down()
    elif move == 'L': game.move_left()
```

```
    elif move == 'R': game.move_right()
    else: continue

    if game.check() == 1:
        print('Game won!')
        break
```